

GitStack Connect - Presentation Outline

Slide 1: Title Slide

- **Title:** GitStack Connect
 - **Subtitle:** Developer Intelligence Platform using Social Network Analysis
 - **Tagline:** “Connecting Developers, Discovering Communities, Recommending Projects”
 - **Your Name/Team**
 - **Date**
-

Slide 2: Problem Statement

- **Challenges:**
 - Developers struggle to find relevant projects
 - Limited visibility into developer communities
 - Difficulty understanding network position
 - Lack of intelligent project matching
 - **Impact:** Wasted time, missed opportunities, disconnected communities
-

Slide 3: Solution Overview

- **GitStack Connect:** A comprehensive developer intelligence platform
 - **Key Capabilities:**
 - Network graph construction from GitHub & StackOverflow
 - Community detection and analysis
 - Personalized repository recommendations
 - Role prediction and profile analysis
 - Interactive network visualization
 - Multi-source project ideas search (GitHub, StackOverflow, Dev.to)
-

Slide 4: System Architecture (High-Level)

- **Three-Tier Architecture:**
 1. **Frontend:** React-based interactive UI
 2. **Backend:** FastAPI REST API
 3. **Data Layer:** SQLite + NetworkX graph
 - **External Integrations:** GitHub API, StackOverflow API, Groq LLM
-

Slide 5: Technology Stack

- **Frontend:**
 - React 19.2.0
 - Cytoscape.js (graph visualization)
 - Axios (API communication)
 - **Backend:**
 - FastAPI (REST API)
 - NetworkX (graph algorithms)
 - Node2Vec (graph embeddings)
 - Sentence Transformers (NLP)
 - Groq LLM (AI features)
 - **Database:** SQLite
-

Slide 6: Knowledge Graph Construction

- **Data Sources:**
 - GitHub: Profiles, repositories, commits, collaborators
 - StackOverflow: Profiles, tags, reputation
 - **Graph Structure:**
 - **Nodes:** Users, Repositories, Tags
 - **Edges:** OWNS, CONTRIBUTES_TO, COLLABORATES_WITH, HAS_TAG
 - **Process:** Fetch → Process → Normalize → Store
-

Slide 7: Network Analysis Algorithms

- **Centrality Metrics:**
 - Degree Centrality (connectivity)
 - Betweenness Centrality (bridge role)
 - Closeness Centrality (reachability)
 - Influence Score (composite metric)
 - **Community Detection:**
 - Louvain Algorithm
 - Modularity optimization
 - Identifies developer clusters
-

Slide 8: Recommendation System Architecture

- **Multi-Stage Pipeline:**
 1. **Candidate Selection:** Profile-based filtering (languages, topics)

2. **Graph-Based Scoring:** Node2Vec embeddings + Jaccard coefficient
 3. **Profile Boost:** Additional scoring for matches
 4. **AI Ranking:** Groq LLM final ranking
- **Result:** Personalized, context-aware recommendations
-

Slide 9: Node2Vec Embeddings

- **Purpose:** Convert graph structure to vector representations
 - **Parameters:**
 - Dimensions: 64
 - Walk Length: 10
 - Number of Walks: 100
 - **Benefits:** Captures structural similarity, enables similarity search
-

Slide 10: Natural Language Processing

- **Topic Extraction:**
 - SBERT embeddings + K-Means clustering
 - Extracts key topics from READMEs and descriptions
 - **Summarization:**
 - Centroid-based extractive summarization
 - Generates 5-6 line professional summaries
 - **Role Classification:**
 - AI-based (primary) + Rule-based (fallback)
 - Multiple role categories
-

Slide 11: AI Integration (Groq LLM)

- **Use Cases:**
 - Professional summary generation
 - Role classification
 - Recommendation ranking
 - **Model:** Llama 3.1 70B Versatile
 - **Benefits:** Context-aware, high-quality outputs
 - **Fallback:** Rule-based methods when API unavailable
-

Slide 12: Key Features - Profile Enrichment

- **GitHub Analysis:**

- Profile information
 - Repository statistics
 - Top languages and technologies
 - Commit activity analysis
 - **StackOverflow Integration:**
 - Reputation and badges
 - Top tags and expertise
 - **AI Summary:** 5-6 line professional summary
-

Slide 13: Key Features - Network Visualization

- **Interactive Graph:**
 - Multiple node types (users, repos, tags)
 - Color-coded edges
 - Community highlighting
 - **Layout Options:** 6 different layouts
 - **Real-time Updates:** Dynamic graph rendering
-

Slide 14: Key Features - Network Metrics

- **Centrality Measures:**
 - Degree, Betweenness, Closeness
 - Influence Score (composite)
 - **Interactive Tooltips:** Explanations for each metric
 - **Visualization:** 2x2 grid display
-

Slide 15: Key Features - Recommendations

- **Personalized Suggestions:**
 - Based on user profile
 - Graph-based similarity
 - AI-powered ranking
 - **Profile-Based Filtering:** Matches languages and topics
 - **Different for Each User:** No duplicate suggestions
-

Slide 16: Key Features - Project Ideas Search

- **User-Driven Search:**
 - Users enter any project idea, technology, or topic

- Example queries: “machine learning”, “React hooks”, “blockchain”, “API design”
 - **Multi-Source Search** (Searches 3 platforms simultaneously):
 - **GitHub Repositories:**
 - * Finds code projects and libraries
 - * Shows: name, description, language, stars, forks
 - * Use: Discover projects to learn from or contribute to
 - **StackOverflow Questions:**
 - * Finds related questions and discussions
 - * Shows: question title, tags, score, answer status
 - * Use: Find problem-solving approaches
 - **Dev.to Articles:**
 - * Finds technical articles and tutorials
 - * Shows: title, author, year, description
 - * Use: Learn technologies and best practices
 - **Features:**
 - Real-time parallel search across all sources
 - Independent pagination for each source
 - Unified results display
 - Direct links to original sources
 - **Use Case:** One-stop search for project ideas, solutions, and learning resources
-

Slide 17: Data Flow Diagram

- **Enrichment Flow:** User Input → Fetch Data → Process → Build Graph → Store
 - **Recommendation Flow:** Profile → Embeddings → Scoring → Ranking → Results
 - **Project Search Flow:** Query → Parallel API Calls (GitHub, SO, Dev.to) → Aggregate → Display
 - **Metrics Flow:** Node ID → Load Graph → Compute → Return
-

Slide 18: Implementation Highlights

- **Database:** SQLite with JSON attributes
 - **Caching:** Embeddings cached for performance
 - **Error Handling:** Graceful fallbacks
 - **API Design:** RESTful endpoints
 - **Code Quality:** Modular, maintainable architecture
-

Slide 19: Results and Outcomes

- Successfully builds knowledge graphs
 - Detects developer communities
 - Provides personalized recommendations
 - Computes network metrics accurately
 - Interactive, user-friendly interface
-

Slide 20: Challenges and Solutions

- **Challenge:** API Rate Limits
 - **Solution:** Caching, polite delays, fallbacks
 - **Challenge:** Large Graph Performance
 - **Solution:** Candidate limiting, efficient algorithms
 - **Challenge:** LLM Quota Exhaustion
 - **Solution:** Quota tracking, rule-based fallbacks
-

Slide 21: Future Improvements

- **Short-Term:**
 - More data sources (GitLab, Bitbucket)
 - Real-time updates
 - Enhanced visualizations
 - **Long-Term:**
 - ML models for prediction
 - Graph database migration
 - Mobile application
 - Advanced analytics
-

Slide 22: Demo / Screenshots

- **Screenshots to Include:**
 1. Main search interface
 2. Profile card with metrics
 3. Network graph visualization
 4. Recommendations panel
 5. Project search results (GitHub, StackOverflow, Research)
 6. Community highlighting
- **Live Demo** (if possible):
 - Search for a user
 - Show network graph
 - Display recommendations

- Explain metrics
-

Slide 23: Technical Achievements

- **Algorithms Implemented:**
 - Node2Vec for embeddings
 - Louvain for community detection
 - Multiple centrality metrics
 - SBERT for NLP
 - **Integration:** Multiple APIs, LLM, graph algorithms
 - **Performance:** Efficient processing, caching
-

Slide 24: Learning Outcomes

- **Skills Developed:**
 - Social Network Analysis
 - Graph algorithms and embeddings
 - NLP and AI integration
 - Full-stack development
 - API integration
 - **Concepts Learned:**
 - Knowledge graphs
 - Recommendation systems
 - Community detection
 - Network metrics
-

Slide 25: Conclusion

- **Summary:** GitStack Connect successfully combines SNA, ML, and AI
 - **Impact:** Helps developers understand their network position and discover projects
 - **Value:** Practical tool with real-world applications
 - **Future:** Scalable foundation for expansion
-

Slide 26: Q&A

- Questions?
- **Contact Information**
- **GitHub Repository** (if public)
- **Thank You!**

Presentation Tips

Visual Elements

- Use diagrams for architecture and data flow
- Include screenshots of the application
- Use color coding for different components
- Show before/after comparisons if applicable

Speaking Points

- **Slide 2-3:** Emphasize the real-world problem
- **Slide 6-10:** Explain algorithms clearly, use examples
- **Slide 11:** Highlight AI integration and fallback mechanisms
- **Slide 17-18:** Showcase technical achievements
- **Slide 21:** Make demo engaging, show real data

Time Allocation (20-minute presentation)

- Introduction (2 min): Slides 1-3
- Architecture & Tech Stack (3 min): Slides 4-6
- Algorithms & Methodologies (5 min): Slides 7-11
- Features & Demo (6 min): Slides 12-17, 22
- Results & Future (3 min): Slides 18-21
- Conclusion & Q&A (1 min): Slides 23-26

Additional Materials

- Prepare backup slides for detailed algorithm explanations
 - Have code snippets ready for technical questions
 - Prepare answers for common questions:
 - Why Node2Vec over other embedding methods?
 - How do you handle large graphs?
 - What's the accuracy of role prediction?
 - How scalable is the system?
-

Key Metrics to Highlight

1. **Performance:**
 - Enrichment: 5-15 seconds
 - Recommendations: 1-3 seconds
 - Real-time graph rendering
2. **Coverage:**
 - Multiple data sources

- Comprehensive profile analysis
 - Community detection
3. **Accuracy:**
- Profile-based filtering
 - AI-powered ranking
 - Context-aware recommendations

Good luck with your presentation!